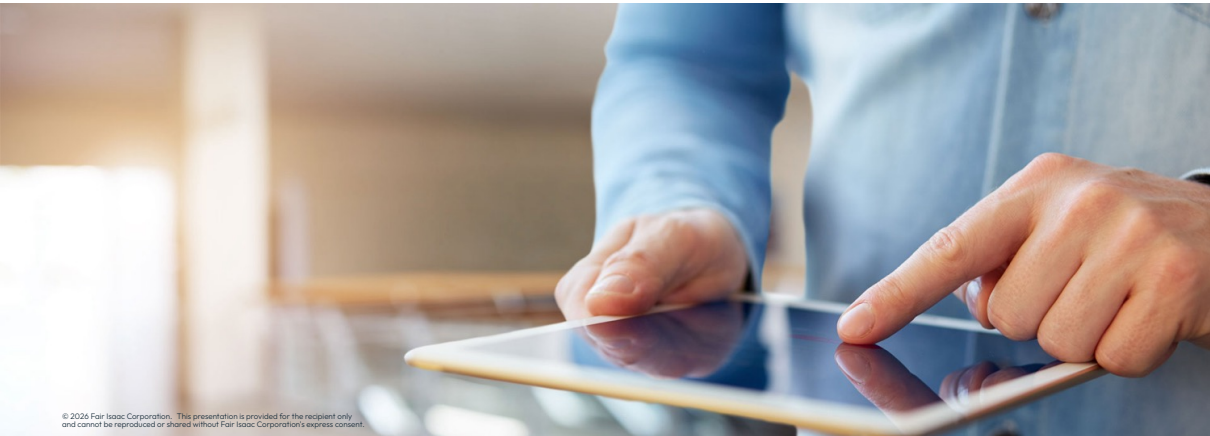
MIP solving with Xpress Optimizer



MIP solving in Xpress

Basic concepts

- IP = Integer Programming
- MIP = Mixed Integer Programming
- MILP = Mixed Integer Linear Programming
  - All used (by us) to mean the same thing!
  - An LP in which some of the variables take integer values

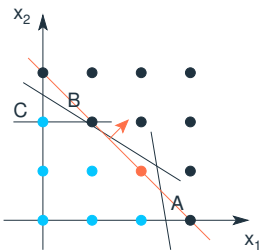
Basic concepts

- IP = Integer Programming
- MIP = Mixed Integer Programming
- MILP = Mixed Integer Linear Programming
  - All used (by us) to mean the same thing!
  - An LP in which some of the variables take integer values

- Allows wealth of decisions to be modeled:
  - Discrete choices: yes/no, on/off, etc
  - Logical conditions: if do A, must do B
  - Discrete quantities: batch sizes
  - Fixed costs
  - Price breaks
  - Piecewise linear functions

Example MIP

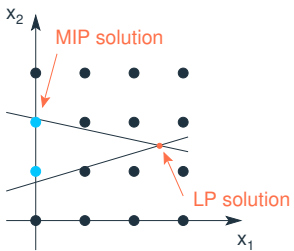
$$
\begin{array}{lrcrcl}
\max & x_1 & + & x_2 & & \\
\text{s.t.} & 6x_1 & + & x_2 & \leqslant 15 & \text{(A)} \\
 & 5x_1 & + & 8x_2 & \leqslant 20 & \text{(B)} \\
 & & & x_2 & \leqslant 2 & \text{(C)} \\
 & x_1, x_2 & \geqslant & 0 & \textbf{and integer} &
\end{array}
$$



$\Rightarrow$ Optimal MIP solution: $x_1 = 2, x_2 = 1$

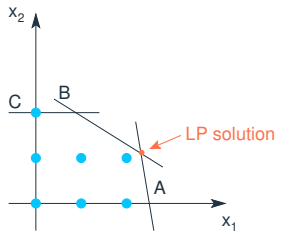
Solution techniques

- Cannot simply round the LP solution:
  - None of the grid points surrounding the 'LP solution' point lie within the feasible region
  - So rounding up or down to the closest integer values will not necessarily result in a MIP solution



LP relaxation

- Relax integer conditions and solve as an LP...



- If the LP solution happens to be integer, then it is the optimal MIP solution
- If the LP is infeasible, then the MIP is infeasible
- In both cases we have “solved“ the MIP problem
- If not... ⇒ Branch & Bound

Presolve

- Presolve attempts to simplify the problem by detecting and removing redundant constraints, tightening variable bounds, etc.
  - Applied before Branch & Bound, which then acts on the presolved problem
  - In some cases, infeasibility may be determined at this stage
- Presolve logging:

```
Original problem has:
       230 rows         2025 cols        12150 elements      1800 entities
Presolved problem has:
       210 rows         1600 cols         9600 elements      1600 entities
Presolve finished in 0 seconds
```

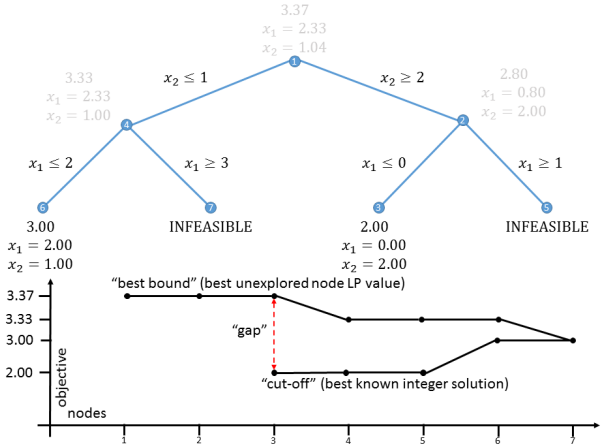
  - Control presolve is ON by default, set to 0 to turn it OFF:

```
p.controls.presolve = 0
```

  - Control the amount of probing during presolve with the preprobing control
  - Perform several rounds of presolving with presolvepasses

Branch and Bound

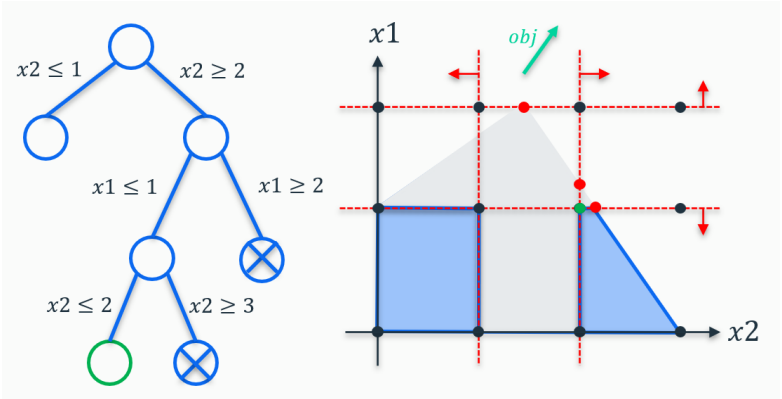
- Example: Branch and Bound for a maximization problem



FICO

Branch and Bound

- How the solver searches for the Branch and Bound tree:
  - Select a 'best node' (by default the best bound, but controllable via the backtrack control)
  - Performs a full dive in (control when to backtrack using the localchoice control)



Heuristics

- The Xpress Optimizer contains a wide variety of heuristics to help find feasible solutions during a MIP solve:
  - Simple rounding heuristics:
    - Fast heuristics that apply some simple rules to the solution of the continuous relaxation in order to produce a feasible MIP solution
    - These are typically run on every node
  - Diving heuristics:
    - Combine rounding and fixing of MIP entities and re-optimize to construct a better quality MIP solution
    - They are run frequently on both the root node and during the branch and bound tree search
    - Specific controls: heurfreq, heurdivestrategy, heurdivespeedup
  - Local search heuristics:
    - The most expensive heuristics and involve solving one or more smaller MIPs whose feasible regions describe a neighborhood around a candidate MIP solution
    - Run during the root cut-loop and typically on every 500 - 1000 nodes during the tree search
    - Specific controls in the next slides

Heuristics

- Important control to experiment with -> heuremphasis control:
  - Try different heuristic strategies that impact the Primal-Dual Integral (PDI)
  - Setting `heuremphasis=1` means more emphasis on finding good solutions early, i.e. on the primal side of the PDI:

```
p.controls.heuremphasis = 1
```

  -1: Automatic selection of heuristic (default)
  0: No heuristics
  1: Focus on reducing the primal-dual gap in the early part of the search
  2: Extremely aggressive search heuristics

Hint: *The value of the PDI can be obtained by querying the problem attribute primaldualintegral*

Large Scale Neighborhood Search (LSNS) heuristics

- Control the overall local search heuristic effort with heursearcheffort:

```
p.controls.heursearcheffort = 2
```

  - A multiplier (type `double`) on the default amount of work the local search heuristics should do
  - A higher value means the local search heuristics will be run more often and that they are allowed to search larger neighborhoods

Large Scale Neighborhood Search (LSNS) heuristics

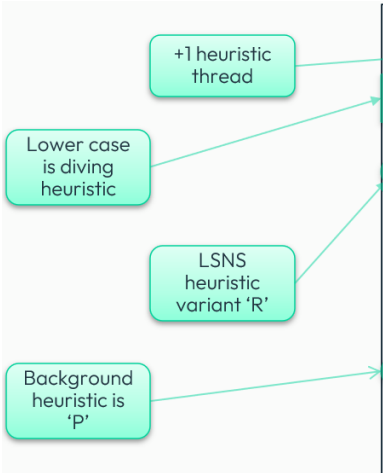
- Control the overall local search heuristic effort with heursearcheffort:

```
p.controls.heursearcheffort = 2
```

  - A multiplier (type `double`) on the default amount of work the local search heuristics should do
  - A higher value means the local search heuristics will be run more often and that they are allowed to search larger neighborhoods
- Other related controls:
  - heurfreq: specifies how many cutting rounds should be done between two runs of the local search heuristic
  - heursearchfreq: for specifying after how many nodes to run the local search again during the tree
  - heursearchrootselect: control local search heuristics to apply on the root node (bit vector)
  - heursearchtreeselect: control local search heuristics to apply during the tree search (bit vector)

Hint: *Heuristic solutions found during the tree search are marked with a letter in front of the log line. A star (*) refers to a MIP-feasible relaxation solution. For more information, check the reference manual page*

MIP Heuristic Logging



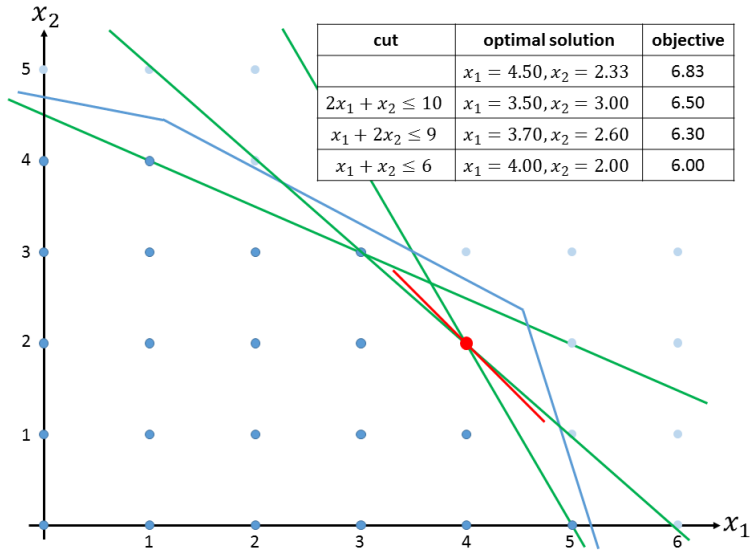

```
Starting root cutting & heuristics
Deterministic mode with up to 1 additional thread

  Its Type    BestSoln     BestBound    Sols    Add    Del     Gap     GInf    Time
a          -2485188.960 -2608070.316       3                   4.71%       0       0
q          -2573127.148 -2608070.316       4                   1.34%       0       0
R          -2573519.443 -2608070.311       5                   1.32%       0       0
R          -2574625.145 -2608070.311       6                   1.28%       0       0
R          -2575017.441 -2608070.311       7                   1.27%       0       0
R          -2586536.868 -2608070.311       8                   0.83%       0       0
R          -2586938.594 -2608070.311       9                   0.81%       0       0
R          -2589181.066 -2608070.311      10                   0.72%       0       0
R          -2589185.326 -2608070.311      11                   0.72%       0       0
   1  K    -2589185.326 -2608070.311      11     27      0     0.72%      16       0
   2  K    -2589185.326 -2608070.310      11     33     20     0.72%      35       0
   3  K    -2589185.326 -2608070.309      11     30     29     0.72%      47       0
   4  K    -2589185.326 -2608070.308      11     32     29     0.72%      45       0
   5  K    -2589185.326 -2608070.307      11     16     28     0.72%      48       0
   6  K    -2589185.326 -2608070.307      11     19     14     0.72%      54       0
   7  K    -2589185.326 -2608070.306      11     48     21     0.72%      52       0
   8  K    -2589185.326 -2608070.305      11     31     46     0.72%      56       0
P          -2590358.518 -2608070.305      12                   0.68%       0       0
P          -2591357.362 -2608070.305      13                   0.64%       0       0
   9  K    -2591357.362 -2608070.305      13     47     31     0.64%      53       0
  10  K    -2591357.362 -2608070.304      13     50     46     0.64%      54       0
P          -2591682.206 -2608070.304      14                   0.63%       0       0
P          -2591953.946 -2608070.304      15                   0.62%       0       0
```

Branch and cut

- A cut is a new constraint, which doesn't change the IP solutions, but cuts off parts of the LP relaxation
- By adding cuts, we strengthen the formulation
- This gives stronger bounds and removes the need for some branching
- But generating cuts takes time, and cuts increase the size of the problem
- So cuts can also slow down optimization

Cut example



cut	optimal solution	objective
	$x_1 = 4.50, x_2 = 2.33$	6.83
$2x_1 + x_2 \leq 10$	$x_1 = 3.50, x_2 = 3.00$	6.50
$x_1 + 2x_2 \leq 9$	$x_1 = 3.70, x_2 = 2.60$	6.30
$x_1 + x_2 \leq 6$	$x_1 = 4.00, x_2 = 2.00$	6.00

Cuts

- Try different levels of automatic cuts with the cutstrategy control:

```
p.controls.cutstrategy = 2
```

  0: cut generation disabled (always try this once)
  1: conservative cut generation strategy
  2: moderate cut generation strategy
  3: aggressive cut generation strategy
 -1: automatic choice between options 1 - 3

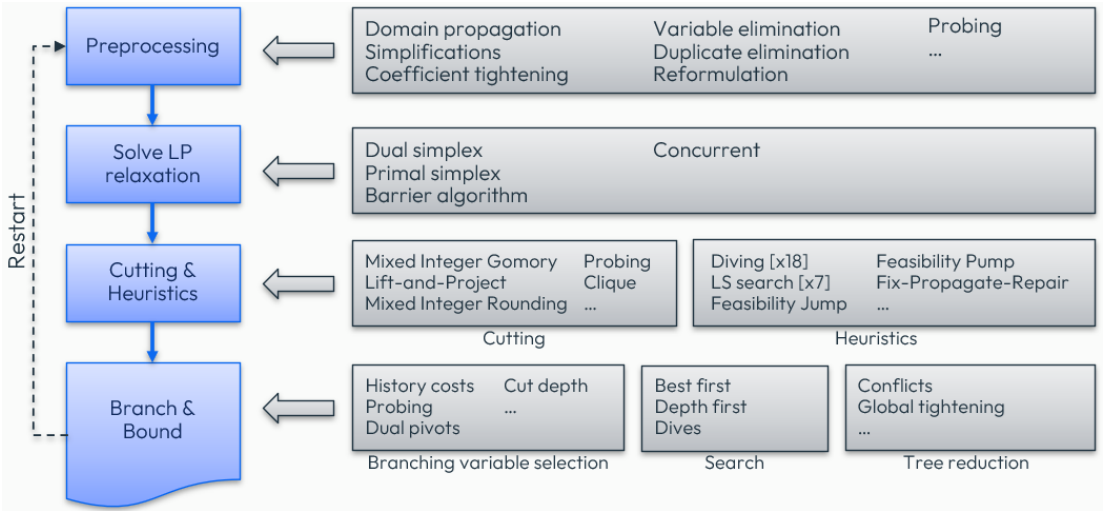
Cuts

- Other controls for determining the amount of in-tree cutting without specifying any specific cut separator:
  - gomcuts and covercuts: number of rounds of Gomory and lifted cover cuts at the root node:
    - There is usually an increased benefit from generating these at the root node, since these inequalities then apply to every subsequent node in the tree search
  - cutselect and treecutselect: number of rounds of lifted cover inequalities at the root node and tree search
    - These are bit vector controls providing detailed management of the cuts created

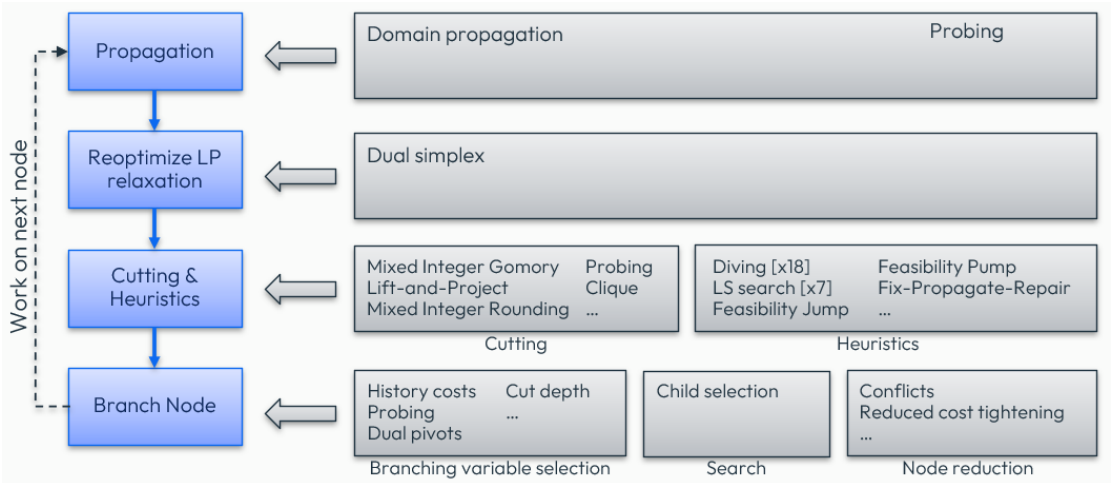
Branch and Cut - Concepts

- Preprocessing: set of routines that eliminates redundant elements and strengthens a model formulation with the aim of accelerating the subsequent solution process (includes presolving)
- Domain propagation: tighten the local domains of variables by inspecting the constraints and current domains of other variables
- Probing: technique that looks at the logical implications of fixing certain variables (*e.g.* fixing a binary variable to 0 and to 1)
- Conflict analysis: generate valid global constraints from infeasible subproblems, derived from cut off subproblems
- Strong branching: precomputes the dual bounds of potential child nodes by solving auxiliary LPs, which leads to smaller search trees

Branch and Cut - Overview



Branch and Cut - Node



Near optimal solutions

- Set stopping criteria if you only require a MIP solution close to optimal:
  - Setting a relative value stopping criterion (example for 1 percent):

```
p.controls.miprelstop = 0.01
```

$$\frac{|\,\text{MIPOBJVAL} - \text{BESTBOUND}|}{\max(|\,\text{BESTBOUND}|,|\,\text{MIPOBJVAL}|)} \leq \text{MIPRELSTOP}$$

  - Setting an absolute value stopping criterion:

```
p.controls.mipabsstop = 100
```

- Set a cut-off if you know a value the MIP solution must beat with mipabscutoff:

```
p.controls.mipabscutoff = 110.0
```

  - The closer the cut-off is to the optimal solution, the quicker the solve will be
  - Setting a cutoff can make a solve slower because the solver might have difficulties to find a first feasible solution
  - If a solution is known it is always better to load it as a start solution instead of setting a cutoff

Optimizer built-in Tuner

- The FICO® Xpress Optimizer Tuner is a tool to automate the process of discovering better control parameter settings:
  - There are over 200 controls affecting solver performance
  - Tuner systematically tests the problem against a range of different combinations of control settings
  - A single tuning run will typically involve solving each problem at least 100-200 times:
    - Can therefore become computationally very expensive for large problems

Optimizer built-in Tuner

- The FICO® Xpress Optimizer Tuner is a tool to automate the process of discovering better control parameter settings:
  - There are over 200 controls affecting solver performance
  - Tuner systematically tests the problem against a range of different combinations of control settings
  - A single tuning run will typically involve solving each problem at least 100-200 times:
    - Can therefore become computationally very expensive for large problems
- Examples of tuner-related controls and functions:

```
p.controls.tunermaxtime = 100      # set max time spent in tuning
p.controls.tunerthreads = 2        # number of concurrent tuner jobs per thread
p.tunerWriteMethod('default.xtm') # export tuner options onto an XTM
p.tunerReadMethod('default.xtm')  # read tuner options from a file
p.tune('g')                        # tune the problem as a MIP
p.optimize()               # optimize the problem with best control settings found
```

Finding help: *Check the Xpress Optimizer tuning guide to learn more about the built-in Tuner*

Warm-start for Barrier

- Barrier can be warm-started in memory using the problem.loadlpsol() method:
  - To accept the given warm-start solution, set the barstart control to -1
- By default, Barrier will use a weighted combination of its own starting point and the provided solution:
  - Use the barstartweight control to experiment with this weighting
- Any provided LP warm-start solution will be used by the Barrier solve of the initial LP relaxation of a MIP:
  - Use the 'l' flag to p.readSlxSol("sol", 'l') for providing a warm-start LP solution using an `.slx` file

Hint: *Check the online Python example that demonstrates warm-starting of Barrier, both using a file and through memory*

Other possibilities for boosting solves

- Add a starting basis with p.loadBasis():
  - Set p.controls.keepbasis=0 to prevent automatic warm-starting
- Use branching priorities with p.loadDirs() to prioritize groups of integer variables:
  - Directives can be given relating to priorities, forced branching directions, pseudo costs and model cuts
- Interact with the solver via callbacks to:
  - Add your own heuristic solutions
  - Tighten node problems
  - Add your own cutting planes / constraints
  - Provide multiple branching candidate for the Solver to choose between
  - Override the Solver branching choice
  - ...

Multi-Threading

- Single machine, shared memory multi-threading:
  - Automatic detection of logical cores available, including container restrictions, with the attribute coresdetected
  - Thread limit set automatically to match cores detected
- Main control: threads, the default number of threads used during optimization
  - Hierarchy of threading controls for fine-grained adjustments with the controls below
- What can be multi-threaded:
  - Dual simplex: use the dualthreads control
  - Barrier algorithm: barthreads and crossoverthreads
  - Concurrent LP/QP: concurrentthreads
  - MIP:
    - Root heuristics: heurthreads and backgroundmaxthreads
    - Branch-and-bound: mipthreads

Predictability and reproducibility in Xpress Solvers

- All solver components are deterministic by default:
  - Running the same problems from the same starting conditions (*e.g.* control settings) will produce identical solves
- Platform independence:
  - win64, linux64 and mac64 on *Intel* will produce identical solves
  - linux64 and mac64 on *ARM* will produce identical solves
- Minimal dependence on threads and architecture:
  - To ensure reproducibility, use the threads control to match number of threads

Note: *Determinism and reproducibility cannot be guaranteed when a user interacts non-deterministically with a solve through callbacks or when setting non-deterministic stopping criteria (e.g. time limit)*

Python Notebook [PN-1]:
Solving and tuning a MIP problem

- Navigate to Exercise 1 - Solving and tuning a MIP problem
- **Exercise 1.1**:
  - Experiment by switching the heuristic emphasis control between 0, 1, and 2 and evaluate the impact in time to optimality
- **Exercise 1.2**:
  - Deactivate cuts by using the cut strategy control and evaluate the impact in time to optimality

Thank You

www.fico.com/optimization